

Αναφορά Εργαστηρίου: Δομές Δεδομένων

Ευστάθιος Μαζγαλτίδης, Μαρία Κούκη

March 25, 2026

1) Συγκριτικός Πίνακας Πολυπλοκότητας

Λειτουργία	Δέντρο (Δυναμικό)	Δέντρο (Array)	Δυαδική Αναζήτηση
insert()	$O(\log N)$	$O(\log N)$	$O(N/2)$
delete()	$O(\log N)$	$O(\log N)$	$O(N/2)$
search()	$O(\log N)$	$O(\log N)$	$O(\log N)$
range()	$O(K + \log N)$	$O(K + \log N)$	$O(K + \log N)$
inorder()	$O(N)$	$O(N)$	$O(1)$

Δέντρα (Δυναμική Παραχώρηση και Array) Οι υλοποιήσεις τους ακολουθούν την ίδια λογική οπότε η πολυπλοκότητα είναι ίδια.

- **Search/Insert/Delete:** Στη μέση περίπτωση, το δέντρο είναι σχετικά ισορροπημένο. Άρα η πολυπλοκότητα εξαρτάται από το ύψος του δέντρου, το οποίο είναι $O(\log N)$.
- **Range Search:** Χρειαζόμαστε $\log N$ βήματα για να εντοπίσουμε την αρχή του εύρους και στη συνέχεια K βήματα για να διατρέξουμε τα K στοιχεία που βρίσκονται εντός του εύρους.
- **Inorder:** Πρέπει υποχρεωτικά να επισκεφθούμε και τους N κόμβους του δέντρου μία φορά, άρα $O(N)$.

Δυαδική Αναζήτηση (σε Ταξινομημένο Πίνακα)

- **Insert/Delete:** Εδώ η απόδοση πέφτει. Παρόλο που βρίσκουμε τη θέση γρήγορα, πρέπει να μετακινήσουμε (shift) τα υπόλοιπα στοιχεία του πίνακα για να διατηρηθεί η ταξινόμηση. Στη μέση περίπτωση μετακινούμε $N/2$ στοιχεία, άρα $O(N/2)$.
- **Search:** Ο αλγόριθμος χωρίζει το διάστημα αναζήτησης στη μέση σε κάθε επανάληψη, οδηγώντας σε $O(\log N)$ συγκρίσεις.
- **Range Search:** Εντοπίζουμε το κάτω όριο με δυαδική αναζήτηση ($O(\log N)$) και μετά διαβάζουμε διαδοχικά τα K στοιχεία του πίνακα μέχρι το πάνω όριο ($O(K)$).
- **Inorder:** Είναι ήδη ταξινομημένο.

2) Χειρότερη Περίπτωση

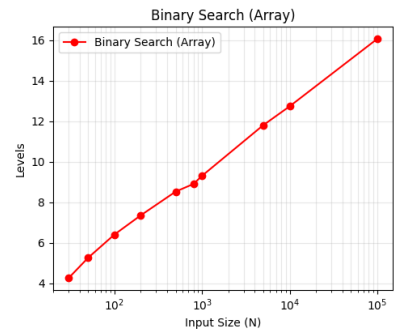
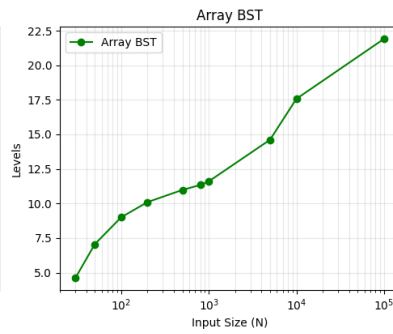
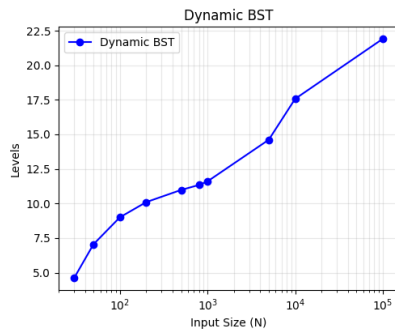
Στη χειρότερη περίπτωση στα δέντρα, αν τα στοιχεία εισαχθούν ήδη ταξινομημένα, το δέντρο παύει να είναι ισορροπημένο και γίνεται σαν linked list. Τότε οι $O(\log N)$ λειτουργίες γίνονται $O(N)$. **Δυαδική Αναζήτηση:** Η αναζήτηση παραμένει $O(\log N)$ στον πίνακα, αλλά η εισαγωγή/διαγραφή γίνεται $O(N)$ επειδή μετακινούμε όλα τα στοιχεία αντί για τα μισά.

3) Γραφήματα και Παρατηρήσεις

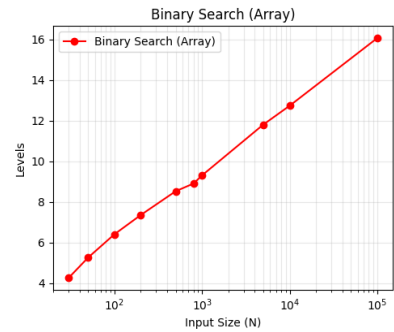
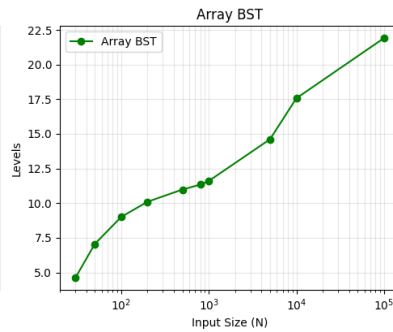
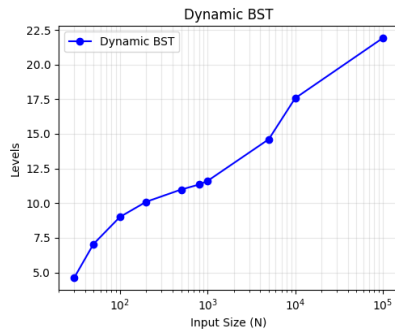
note Τα γραφήματα έγιναν σε ξεχωριστά subplots για την καλύτερη παρατήρηση των τιμών.

— INSERT —

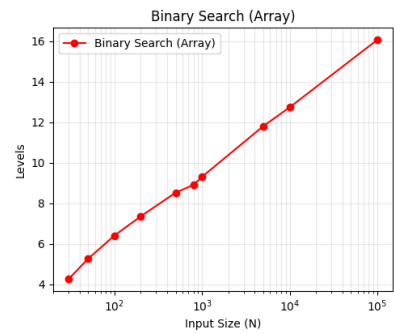
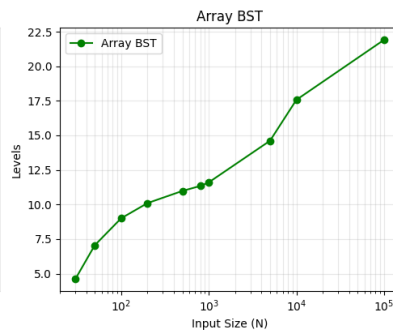
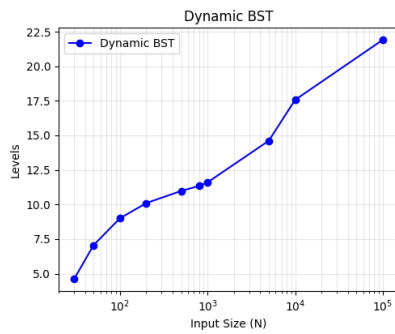
Operation Search: Levels



Operation Search: Levels



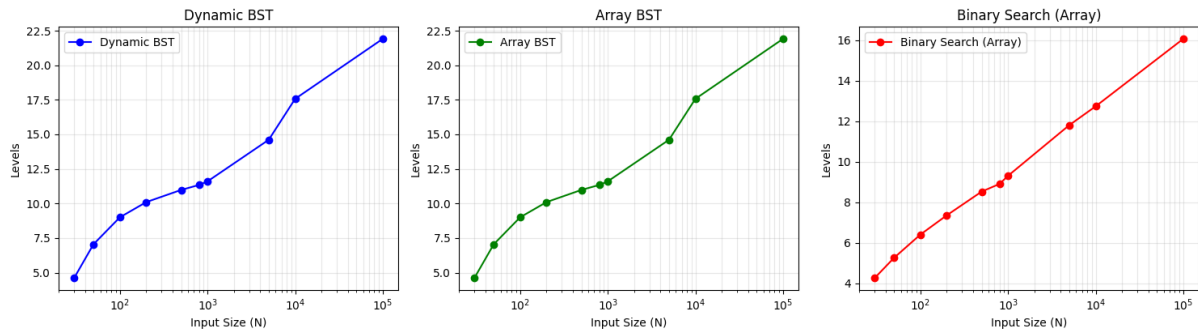
Operation Search: Levels



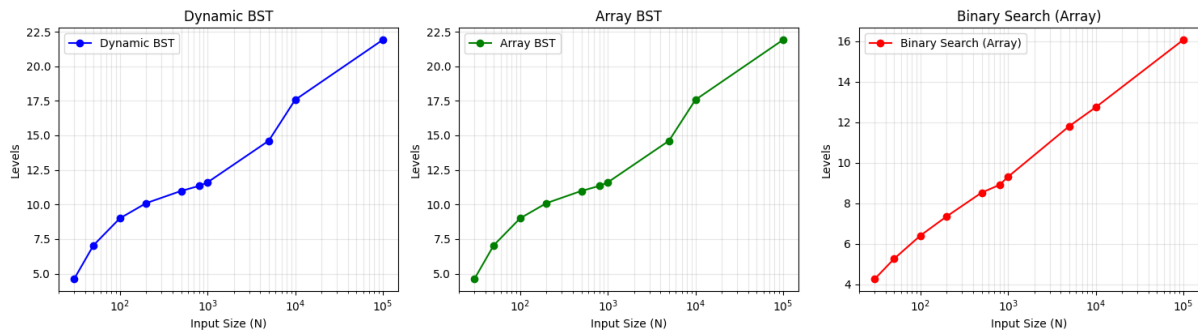
Οι πράξεις, ο χρόνος και τα επίπεδα, για δέντρο με δυναμική παραχώρηση μνήμης και για δέντρο με array, έχουν λογαριθμική μορφή με ελάχιστα μεγαλύτερη τιμή από τη θεωρητική. Στη δυαδική αναζήτηση ακολουθεί τη γραμμική εξέλιξη που περιμένουμε και από τη θεωρία.

— SEARCH —

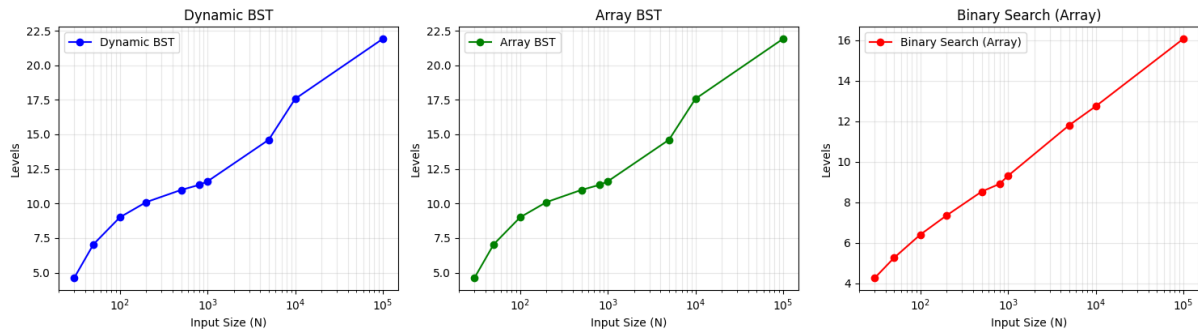
Operation Search: Levels



Operation Search: Levels



Operation Search: Levels



Και στις τρεις δομές, παρατηρείται καθαρά λογαριθμική εξέλιξη όπως και στην θεωρία.

4) Ανάλυση Μεγάλων Δεδομένων

Στα μεγάλα N οι χρόνοι ακολουθούν την τάση των πράξεων, επιβεβαιώνοντας τη θεωρητική αναλογία μεταξύ τους. Στη δυαδική αναζήτηση η αύξηση είναι ανάλογη και στα δέντρα ο χρόνος παραμένει σε χαμηλά επίπεδα ακολουθώντας τη λογαριθμική καμπύλη των πράξεων. Η παρατήρηση βγάζει νόημα, καθώς με την αύξηση του μεγέθους της εισόδου η πολυπλοκότητα του αλγορίθμου κυριαρχεί έναντι των δευτερευουσών καθυστερήσεων του συστήματος.

5) Συμπεράσματα Υλοποίησης

Θεωρητικά η υλοποίηση με πίνακα αναμένεται ταχύτερη λόγω καλύτερης τοπικότητας μνήμης. Στις μετρήσεις μας όμως, αν και οι πράξεις και τα επίπεδα είναι ταυτόσημα, η δυναμική παραχώρηση αποδείχθηκε ελαφρώς

ταχύτερη σε απόλυτους χρόνους. Συμπερασματικά, η υλοποίηση με δυναμική παραχώρηση υπερτερεί στην πράξη, πιθανώς λόγω πιο αποδοτικής διαχείρισης αντικειμένων από τη Java σε σύγκριση με την πρόσβαση σε μεγάλους πίνακες που απαιτούν συνεχείς ελέγχους.

Σημείωση Υλοποίησης

Η υλοποίηση του δυαδικού δέντρου αναζήτησης με δυναμική παραχώρηση μνήμης βασίστηκε σε κώδικα από το φροντιστηριακό υλικό του μαθήματος. Η αρχική αυτή υλοποίηση προσαρμόστηκε κατάλληλα ώστε να καλύπτει πλήρως τις απαιτήσεις της άσκησης, υποστηρίζοντας τις λειτουργίες `insert`, `delete`, `search`, `rangeSearch` και `inorder`, καθώς και τη συλλογή των μετρήσεων που χρησιμοποιήθηκαν στο πειραματικό μέρος.

Δεδομένα Μετρήσεων

N = 30

BST with Dynamic Memory Inorder: 2 8 10 13 14 15 18 20 23 24 25 26 28 32 39 40 41 42 43 44 45 47 48 49 51 54 55 58 59 60

BST Array Inorder: 2 8 10 13 14 15 18 20 23 24 25 26 28 32 39 40 41 42 43 44 45 47 48 49 51 54 55 58 59 60

Operation A - Insert

N	DynOps	DynTime(ns)	DynLev	ArrOps	ArrTime(ns)	ArrLev	BinOps	BinTime(ns)	BinLev
30	13.80	1779.90	5.10	13.80	2032.40	5.10	14.90	2569.00	6.45
50	18.75	2009.15	7.40	18.75	2442.70	7.40	28.70	3784.75	8.35
100	27.15	2411.00	10.15	27.15	2954.95	10.15	37.90	4528.15	9.30
200	25.10	2518.25	10.00	25.10	3048.30	10.00	68.70	7627.95	11.65
500	27.62	2684.52	10.76	27.62	2809.82	10.76	166.56	15009.24	13.58
800	28.18	2037.90	11.24	28.18	2506.26	11.24	232.70	19330.20	14.28
1000	30.66	2438.48	12.14	30.66	2944.38	12.14	354.74	31765.46	15.14
5000	37.48	2829.46	14.43	37.48	3628.88	14.43	1349.95	107738.30	18.50
10000	44.59	2997.74	17.63	44.59	3700.65	17.63	2584.25	213671.18	19.36
100000	52.51	3691.64	20.82	52.51	4387.02	20.82	26293.41	1433685.03	24.46

Operation B - Delete

N	DynOps	DynTime(ns)	DynLev	ArrOps	ArrTime(ns)	ArrLev	BinOps	BinTime(ns)	BinLev
30	11.40	2251.25	6.10	11.40	3101.30	6.10	18.05	3000.25	4.50
50	14.80	2104.00	7.80	14.80	2870.30	7.80	26.20	3377.40	5.45
100	15.05	2037.40	7.80	15.05	2589.30	7.80	33.45	4829.05	6.25
200	23.00	3066.40	11.65	23.00	3659.90	11.65	51.00	5763.45	7.65
500	21.26	2709.12	11.00	21.26	3459.94	11.00	170.94	18167.78	8.26
800	23.54	2484.26	12.08	23.54	3173.00	12.08	259.00	24951.66	9.18
1000	23.12	2343.72	11.88	23.12	3063.78	11.88	290.10	28526.12	9.56
5000	30.06	3067.41	15.48	30.06	3970.97	15.48	1751.09	158992.90	11.78
10000	34.66	3031.08	17.70	34.66	3993.63	17.70	2900.97	266254.73	12.75
100000	44.05	3668.09	22.35	44.05	4667.56	22.35	25223.45	1506464.55	16.12

Operation C - Search

N	DynOps	DynTime(ns)	DynLev	ArrOps	ArrTime(ns)	ArrLev	BinOps	BinTime(ns)	BinLev
30	8.80	1206.35	4.60	8.80	1462.25	4.60	8.10	1382.05	4.25
50	13.50	1374.55	7.05	13.50	1760.90	7.05	9.90	1425.80	5.25
100	17.55	1674.15	9.00	17.55	2111.35	9.00	12.35	1700.30	6.40
200	19.80	2644.95	10.10	19.80	2378.40	10.10	14.30	1899.00	7.35
500	21.50	2009.22	10.98	21.50	2357.76	10.98	16.58	1941.68	8.52
800	22.12	1702.82	11.34	22.12	2223.34	11.34	17.24	1884.34	8.90
1000	22.70	2005.20	11.60	22.70	2471.18	11.60	18.10	2253.76	9.30
5000	28.76	2167.96	14.61	28.76	2999.90	14.61	23.14	2477.54	11.80
10000	34.58	2592.55	17.57	34.58	3353.73	17.57	24.90	2621.48	12.73
100000	43.30	3080.26	21.91	43.30	3895.32	21.91	31.60	2120.00	16.06

Operation D - Range Search

N	DynOps	DynTime(ns)	DynLev	ArrOps	ArrTime(ns)	ArrLev	BinOps	BinTime(ns)	BinLev
30	29.55	14299.85	9.85	29.55	8623.85	9.85	10.40	6327.70	10.40
50	62.10	15235.75	20.70	62.10	17845.15	20.70	20.95	10851.95	20.95
100	129.30	33276.10	43.10	129.30	38529.15	43.10	42.70	23197.80	42.70
200	206.25	64582.80	68.75	206.25	72211.35	68.75	67.35	47262.15	67.35
500	479.52	138311.48	159.84	479.52	144135.18	159.84	159.56	97409.00	159.56
800	854.76	223127.04	284.92	854.76	250051.18	284.92	284.56	158017.02	284.56
1000	1017.90	269821.52	339.30	1017.90	303997.70	339.30	338.62	192692.82	338.62
5000	5160.57	1302607.71	1720.19	5160.57	1484380.08	1720.19	1719.38	932982.49	1719.38
10000	10417.02	2484817.87	3472.34	10417.02	2835889.54	3472.34	3469.07	1730876.40	3469.07
100000	104430.54	16977243.10	34810.18	104430.54	19455782.43	34810.18	34806.35	11710943.60	34806.35